

InfusioTech Restaurant Ordering SaaS

Product Requirements Document (PRD) | Updated Edition

Version 1.1 | 2 October 2026

Prepared for InfusioTech

Document purpose

This PRD defines the product scope, user experience, functional requirements, access-control model, architecture and MVP acceptance criteria. This edition specifies a single restaurant mobile app for owners, managers and staff, including owner-managed staff invitations and configurable permissions.

1. Executive Summary

InfusioTech Restaurant Ordering SaaS is a multi-tenant platform that enables restaurants to accept direct digital orders through table-specific QR codes, manage operations from a shared restaurant mobile app, and monitor business performance. Customers use a responsive web experience and do not need to install an app.

The restaurant owner and authorized staff use one Android/iOS mobile app with role-based access. Owners can manage the restaurant and delegate selected permissions to staff members from the app. A web admin dashboard may be provided as an optional companion interface, but it is not required for owner operations.

2. Product Vision and Goals

Vision: Give independent restaurants a simple, affordable direct-ordering and operations system without requiring customers to download an app.

Primary goals: Enable QR-based ordering; centralize live order handling; let owners manage staff and permissions; provide menu, table, customer and analytics tools; support multiple restaurant tenants securely; create a subscription-based SaaS business.

Success indicators: Restaurant activation rate; orders successfully submitted; order processing time; monthly active restaurants; subscription renewal; payment reconciliation accuracy; staff onboarding completion; support-ticket rate.

3. Users and Roles

Role	Purpose	Typical access
Customer	Browse and order	Menu, cart, checkout, order status, review link
Owner	Business owner / account administrator	Full tenant access, subscription, staff, permissions, settings, reports
Manager	Delegated day-to-day lead	Owner-selected operational and management permissions
Waiter / Service Staff	Handle guest and table operations	Assigned orders, tables, order status
Cashier	Handle checkout and payment records	Orders and payment status, permitted reconciliation
Kitchen Staff	Prepare incoming orders	Kitchen order view, preparing/ready status
InfusioTech Super Admin	Operate SaaS platform	Tenant support, subscription administration, platform configuration

4. Product Scope

MVP in scope: Restaurant onboarding; tenant profile; menu and category management; table management; table QR generation; customer web ordering; cart and checkout; cash and online payment options; live order queue; order status lifecycle; one restaurant mobile app for owners and staff; staff invitations and role-based permissions; basic customer order history; basic analytics; platform-level tenant and subscription management.

Later releases: Multi-branch operations; advanced analytics; loyalty and offers; integrations with POS/accounting/delivery platforms; advanced inventory; automated marketing; additional languages; custom domain and deeper branding.

Out of scope for MVP: Customer native mobile app; proprietary payment processing; hardware manufacturing; delivery fleet management; full POS replacement; payroll and HR suite.

5. Core Customer Experience

- Customer scans a table-specific QR code or taps an NFC tag.
- The restaurant-branded responsive menu opens in a mobile browser/PWA.
- Customer browses categories, item details, images, variants, add-ons and availability.
- Customer adds items to cart, confirms table/order details and chooses an enabled payment method.
- The backend validates prices, taxes, availability and totals before creating the order.
- Customer receives an order confirmation and can view the current order status.
- After service, the restaurant may show its configured review link.

6. Restaurant Mobile App

A single Flutter application for Android and iOS serves owners, managers and staff. The app renders navigation and actions according to the signed-in user's tenant, role and explicitly granted permissions.

Shared operational capabilities: Secure login; assigned restaurant selection if user belongs to multiple tenants; live order list; order details; accept/reject where authorized; preparing/ready/served transitions where authorized; table status; payment state visibility; push notifications; profile and password/session controls.

Owner capabilities: Full access to restaurant configuration; menu categories and items; prices, variants and add-ons; operating hours, tax and payment settings; tables and QR/NFC associations; staff invitation and account lifecycle; role templates and per-user permission overrides; customer records and order history; sales and item analytics; subscription and billing information; audit/activity history.

Manager capabilities: Access to selected owner functions as granted, such as order operations, menu editing, table management, staff coordination or reports. Manager does not automatically receive owner-only controls.

Staff capabilities: Only assigned operational functions. Staff access is individually configurable and may be further constrained by role template, branch, shift or assigned work area in later releases.

Mobile UX requirements: Clear role-aware home screen; fast order queue; visible status and payment indicators; confirmation for destructive or sensitive actions; accessible touch targets; graceful offline/poor-network states; push notification preferences.

7. Staff Management and Role-Based Access Control (RBAC)

Owners manage staff directly from the restaurant mobile app. Access is granted per restaurant tenant and may be based on a role template with owner-configurable permission overrides.

- Owner opens Staff Management and chooses Add Staff / Invite Staff.
- Owner enters staff name and contact information, selects a role template and optionally adjusts individual permissions.
- System sends a secure invitation or one-time activation link. Staff sets their own authentication credential; passwords are never shared by the owner.
- Staff accepts the invitation and is linked to the correct restaurant tenant.
- Owner can edit permissions, suspend/reactivate an account, or revoke access. Changes take effect on the server and active sessions are refreshed or revoked as appropriate.

Permission examples: View orders; accept orders; reject/cancel orders; update preparation status; mark served; view tables; edit tables; view payment status; record cash received; reconcile payments; view menu; create/edit menu items; change prices; manage availability; view customer records; export customer data; view basic/advanced analytics; manage staff; manage restaurant settings; manage subscription.

Role defaults: Owner: all tenant permissions. Manager: selected operational and management permissions. Waiter: assigned orders/tables and status updates. Cashier: order and payment visibility plus permitted cash reconciliation. Kitchen Staff: kitchen queue and preparation status. Defaults are templates only; the owner may customize them, subject to protected owner-only permissions.

Security rules: Enforce permissions on backend APIs and database policies, not only by hiding UI controls. Deny by default. Every staff identity must be individually attributable. Do not share owner credentials. Restrict staff management, ownership transfer, tenant deletion and subscription ownership to the owner or explicitly authorized platform process. Record sensitive actions in an audit log.

Acceptance criteria: Owner can invite a staff member; invited user can activate account; staff sees only permitted screens/actions; API requests without permission are rejected; owner can change/revoke access; revoked users cannot continue using protected APIs; actions record actor, tenant, timestamp and relevant change.

8. Restaurant Web Admin Dashboard (Optional Companion)

The owner's core administration tasks must be possible in the mobile app. An optional responsive web dashboard may provide a larger-screen interface for menu bulk editing, exports, reporting and configuration. It uses the same tenant, roles and permissions as the mobile app and does not create a separate permission system.

9. Order and Payment Management

Order lifecycle: New → Accepted → Preparing → Ready → Served. Authorized users may reject or cancel with a reason. Transitions are validated server-side and recorded.

Payment lifecycle: Unpaid / Pending → Paid, Failed, Refunded or Cash Received / Reconciled, as applicable. Payment state is separate from order state.

Rules: Server recalculates item prices, variants, discounts, taxes and totals. Store an immutable order-item/price snapshot. Verify online payment through signed gateway webhooks or server-side verification. Use idempotency for payment and order callbacks. Never treat order creation as proof of payment. Record cash receipt separately from online settlement.

10. Menu, Tables and Restaurant Configuration

Menu: Categories, items, descriptions, images, prices, variants, add-ons, availability, dietary labels and item ordering.

Tables and QR: Create/edit/disable tables; generate unique table-specific QR URLs; map NFC tags to the same secure destination; reissue codes; prevent cross-tenant table mapping.

Restaurant settings: Name, address, logo, brand colors, contact details, operating hours, order acceptance settings, taxes, enabled payment methods, review URL and notification preferences.

11. CRM and Analytics

CRM: Customer contact details where voluntarily supplied, order history, order frequency and spend summaries. Collect only necessary data and provide retention/deletion controls consistent with applicable requirements.

MVP analytics: Orders by day; gross sales; online vs cash payment summary; popular items; average order value; order status counts; date filters.

Access: Analytics and customer-data access require explicit permissions. Export functionality should be separately permissioned and logged.

12. InfusioTech Super Admin

Platform operators manage the SaaS service across tenants. Capabilities include tenant search and status; plan/subscription administration; support workflows; platform configuration; usage and service health; onboarding assets such as QR/NFC kits. Support access to restaurant data should be limited, justified, time-bound where feasible and audited. Super Admin is not a restaurant owner role.

13. Subscription Plans and Commercial Model

Plan	Indicative price	Illustrative features
Starter	■2,999/year	QR menu, direct ordering, customer web flow, basic order dashboard and reports
Growth	■5,999/year	Starter plus restaurant mobile app, live updates, tables, CRM and item/sales analytics
Premium	■9,999/year	Growth plus advanced analytics, granular staff roles, multi-branch options and priority support

These are proposed positioning prices, not verified market quotations. Confirm plan entitlements before launch. NFC printing/kit, payment gateway charges, messaging charges, custom integrations and exceptional support may be billed separately. Subscription status and entitlement checks must be enforced server-side.

14. Technical Architecture

Customer experience: Responsive web app/PWA using Next.js/React and Tailwind; no customer app install required.

Restaurant app: Flutter for Android and iOS with role-aware navigation and screens.

Backend/data: Supabase PostgreSQL, Auth, Storage, Realtime and Edge Functions or equivalent managed backend; server-side APIs for privileged actions.

Hosting and integrations: Vercel or equivalent for web hosting; Razorpay or another supported gateway; push notification provider; email/SMS/WhatsApp provider as separately configured.

Multi-tenancy: Every restaurant-owned record includes tenant_id. Enforce tenant isolation with database Row Level Security and backend authorization. Validate tenant membership on every request. Avoid trusting tenant identifiers supplied by the client.

15. Core Data Model (High Level)

Entities include Tenant/Restaurant; Subscription/Plan; User; TenantMembership; Role; Permission; RolePermission; UserPermissionOverride; StaffInvitation; Branch (future); Table; QR/NFC Asset; MenuCategory; MenuItem; Variant; AddOn; Customer; Order; OrderItemSnapshot; Payment; Refund; Notification; AuditEvent.

Membership links a user to a tenant. Effective permissions are calculated from protected role defaults plus authorized overrides. Orders and operational records are tenant-scoped. Sensitive records should have explicit retention and access rules.

16. Security, Privacy and Compliance

Authentication: Secure authentication, verified invitation flow, session expiry/refresh, optional MFA for owners and platform admins.

Authorization: Least privilege, deny-by-default RBAC, server-side checks and database RLS; tenant scoping for every query and storage object.

Payments: Gateway verification, signed webhook validation, idempotency, no storage of raw card credentials.

Auditability: Log staff invitations, role/permission changes, menu price edits, refunds, payment reconciliation, exports, sensitive settings and admin support access.

Privacy: Data minimization, purpose limitation, retention/deletion policy, access controls, secure transport and storage, and appropriate notices/consents. Review applicable Indian privacy, tax and consumer requirements with qualified advisers before commercial launch.

Operational safeguards: Backups, restore testing, rate limiting, monitoring, error tracking, secrets management and incident response procedure.

17. Non-Functional Requirements

Performance: Common mobile and customer flows should feel responsive on typical Indian mobile networks; use pagination and optimized images.

Availability: Production uptime and recovery objectives must be defined against selected hosting plans; free tiers are for prototype/pilot and do not guarantee production capacity.

Scalability: Support tenant growth through indexed tenant-scoped queries, background jobs where required and observable resource limits.

Usability: Simple onboarding, clear status labels, readable screens, accessible contrast and low-friction staff activation.

Reliability: Order/payment state consistency, retry-safe webhooks, graceful network recovery and monitored notifications.

18. MVP Delivery Phases

- Phase 1 — Foundation: tenant onboarding, authentication, tenant isolation, menu, tables and QR links.
- Phase 2 — Customer ordering: customer web flow, cart, server-validated order creation, payment integration and confirmation.
- Phase 3 — Restaurant operations: Flutter app, live order queue, notifications, order lifecycle and payment visibility.
- Phase 4 — Owner and staff controls: owner mobile administration, staff invitations, role templates, custom permissions, revocation and audit events.
- Phase 5 — Insights and launch: basic CRM/analytics, super-admin controls, subscription entitlements, QA, pilot and production hardening.

19. Acceptance and Launch Checklist

- Tenant A cannot access Tenant B's menu administration, customers, tables, orders, files or reports.
- Customer can scan a table QR and place an order without installing an app.
- Restaurant can receive and update an order from the mobile app.
- Owner can complete core administration from the mobile app.
- Owner can invite staff and customize their permissions.
- Unauthorized staff API actions are denied even if manually invoked.
- Owner can suspend/revoke staff access and changes take effect promptly.
- Order status and payment status remain distinct and auditable.
- Payment verification, duplicate webhook handling and failure cases are tested.
- Backups, monitoring, support workflow, privacy notices and subscription handling are ready.

20. Risks, Assumptions and Open Decisions

Risks: Permission complexity; unreliable restaurant connectivity; payment reconciliation errors; tenant data leakage; notification delays; adoption friction; hosting costs as usage grows.

Assumptions: Restaurants have internet-connected Android/iOS devices; customers can use a mobile browser; each restaurant is a tenant; owner is the primary tenant administrator.

Decisions before build: Final plan entitlements and pricing; whether staff login uses phone, email or both; whether staff can belong to multiple tenants; whether branch-level permissions are MVP or later; gateway and notification providers; tax and invoice rules; data retention period; cancellation/refund workflow.